



Algorithms

Linear-Time Sorting Algorithms

Mid Term Syllabus

- Basics of Algorithm
- Sorting- Heap, merge
- Linear Sorting- Counting, Radix and bucket
- Graph Algorithm- BFS, DFS, Connected components, topological ordering, cut vertex, bridge
- MST

Sorting So Far

- Insertion sort:
 - Easy to code
 - Fast on small inputs (less than ~50 elements)
 - Fast on nearly-sorted inputs
 - $O(n^2)$ worst case
 - $O(n^2)$ average (equally-likely inputs) case
 - $O(n^2)$ reverse-sorted case

Sorting algo

stable
sort

inplace sort

✓✓

extra space req

for merge sort $= O(1)$

extra space $= O(n)$

Sort not inplace

10 20 30 5 10 50
A

5 10 10 20 30 50
A B

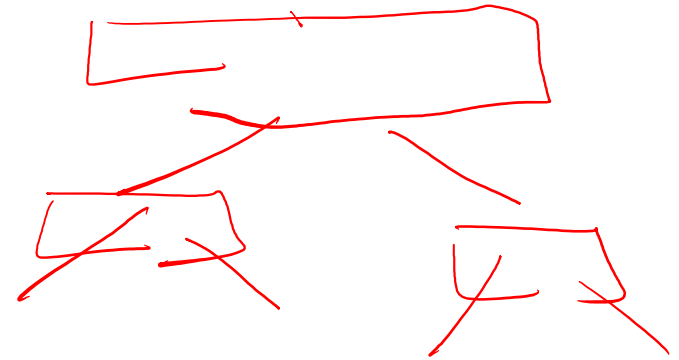
Selection X

heap X

merge, Bubble and insertion ✓

Sorting So Far

- Merge sort:
 - Divide-and-conquer:
 - Split array in half
 - Recursively sort subarrays
 - Linear-time merge step
 - $O(n \lg n)$ worst case
 - Doesn't sort in place



$$T(n) = 2T(n/2) + \text{merge}$$

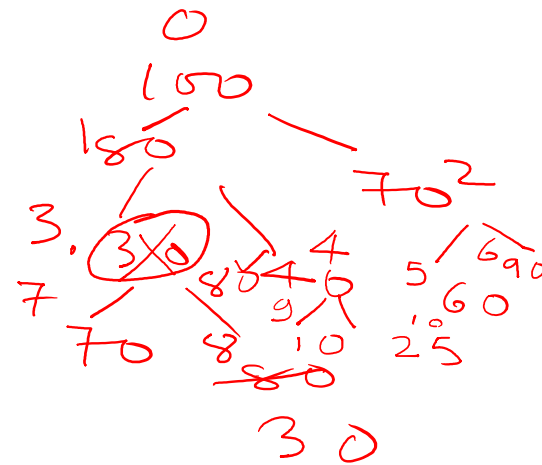
$$T(n) = kT(n/k) + kn$$

$$T(n) = n \log_2 n$$


heapify (a, 3)

max heap

$O(n)$
heapify (a, i)
Build heap (a, n)
insert key (a, key)
extract_max
heap sort



Sorting So Far

□ Heap sort:

- Uses the very useful heap data structure

 - Complete binary tree

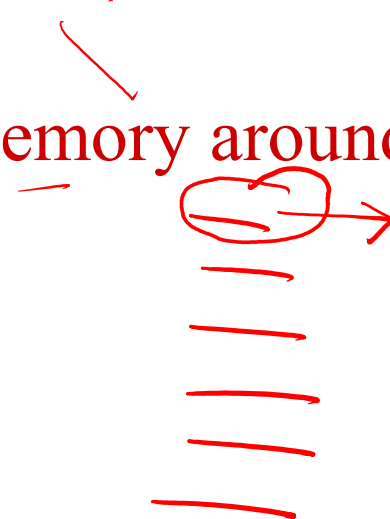
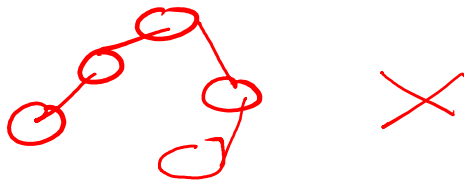
 - Heap property: parent key $>$ children's keys

= almost complete binary tree

- $O(n \lg n)$ worst case

- Sorts in place

- Fair amount of shuffling memory around



heap n

{ Build-heap (a, n)

For $i = n/2$ to 1
 heapify (a, i)

}

Sorting So Far

- Quick sort:
 - Divide-and-conquer:
 - Partition array into two subarrays, recursively sort
 - All of first subarray $<$ all of second subarray
 - No merge step needed!
 - $O(n \lg n)$ average case
 - Fast in practice
 - $O(n^2)$ worst case
 - Naïve implementation: worst case on sorted input
 - Address this with randomized quicksort

Sorting In Linear Time

- Counting sort

- No comparisons between elements!
- **But**...depends on assumption about the numbers being sorted
 - We assume numbers are in the range $1..k$
- The algorithm:
 - Input: $A[1..n]$, where $A[j] \in \{1, 2, 3, \dots, k\}$
 - Output: $B[1..n]$, sorted (notice: not sorting in place)
 - Also: Array $C[1..k]$ for auxiliary storage

- Counting sort determines, for each input element x , the number of elements less than x
- It uses above information to place element x directly into its position in the output array.
- This algo requires two additional arrays
 $B[1, \dots, n]$ as output array, and $C[0, 1, \dots, k]$
provides temporary working space

Counting Sort

```
1    CountingSort(A, B, k)
2        for i=0 to k
3            C[i]= 0;
4        for j=1 to n
5            C[A[j]] += 1;
6        for i=1 to k
7            C[i] = C[i] + C[i-1];
8        for j=n downto 1
9            B[C[A[j]]] = A[j];
10       C[A[j]] -= 1;
```

- In line no 5, if the value of input elt. is 'i' then we increment $C[i]$ by 1.
- After line 5, $C[i]$ will contain the no of elts equal to i .
- Line 6 and 7 determines for each $i=0,1,\dots,k$, how many input elements are less than or equal to 'i' by keeping a running sum of the array C

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

(a)

	0	1	2	3	4	5
C	2	2	4	7	7	8

(b)

	1	2	3	4	5	6	7	8
B							3	

	0	1	2	3	4	5
C	2	2	4	6	7	8

(c)

	1	2	3	4	5	6	7	8
B		0					3	

	0	1	2	3	4	5
C	1	2	4	6	7	8

(d)

	1	2	3	4	5	6	7	8
B		0				3	3	

	0	1	2	3	4	5
C	1	2	4	5	7	8

(e)

	1	2	3	4	5	6	7	8
B	0	0	2	2	3	3	3	5

(f)

Counting Sort

```
1  CountingSort(A, B, k)
2      for i=1 to k
3          C[i]= 0;
4      for j=1 to n
5          C[A[j]] += 1;
6      for i=2 to k
7          C[i] = C[i] + C[i-1];
8      for j=n downto 1
9          B[C[A[j]]] = A[j];
10     C[A[j]] -= 1;
```

Takes time $O(k)$

Takes time $O(n)$

What will be the running time?

Counting Sort

- Total time: $O(n + k)$
 - Usually, $k = O(n)$
 - Thus counting sort runs in $O(n)$ time
- But sorting is $\Omega(n \lg n)$!
 - No contradiction--this is not a comparison sort (in fact, there are *no* comparisons at all!)
 - Notice that this algorithm is *stable*

Counting Sort

- Cool! *Why don't we always use counting sort?*
- Because it depends on range k of elements
- *Could we use counting sort to sort 32 bit integers? Why or why not?*
- Answer: no, k too large ($2^{32} = 4,294,967,296$)

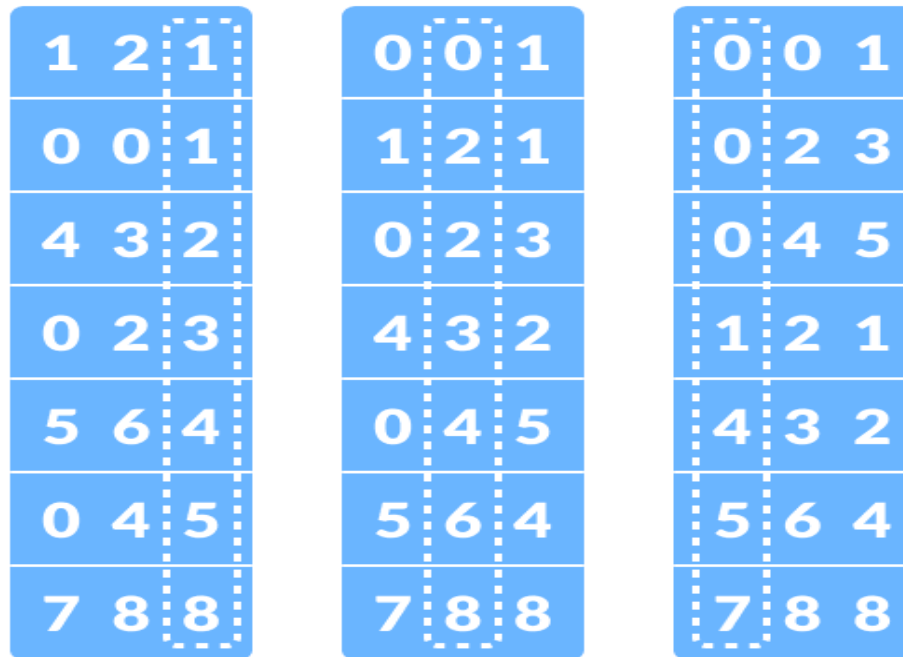
Radix Sort

- Key idea: sort the *least* significant digit first

```
RadixSort(A, d)
```

```
    for i=1 to d
```

```
        StableSort(A) on digit i
```



1	2	1
0	0	1
4	3	2
0	2	3
5	6	4
0	4	5
7	8	8

0	0	1
1	2	1
0	2	3
4	3	2
0	4	5
5	6	4
7	8	8

0	0	1
0	2	3
0	4	5
1	2	1
4	3	2
5	6	4
7	8	8

sorting the integers according to units, tens and
hundreds place digits

Radix Sort

- Sort n numbers on digits that range from $1..k$
- Time: $O(n + k)$
- Each pass over n numbers with d digits takes time $O(n+k)$, so total time $O(dn+dk)$
 - When d is constant and $k=O(n)$, takes $O(n)$ time
- *How many bits in a computer word?*

Radix Sort

- Radix sort is a non-comparative integer sorting algorithm that sorts numbers based on their individual digits or radix positions

Radix Sort

- *Can radix sort be used on floating-point numbers?*
- Floating-point numbers have limited precision. The fractional part (mantissa) can be quite long, which means that the number of possible values in each radix position can be extremely large.

- 
- This can make radix sort less efficient for floating-point numbers compared to integers, as you may need to sort a large number of buckets at each radix position.
 - Radix sort can be used on floating-point numbers, but it requires some adaptations compared to its typical usage with integers.

- 
- 1. Convert floating-point numbers to their binary representations: Floating-point numbers are typically represented in IEEE 754 format, which includes a sign bit, an exponent, and a mantissa (fractional part).
 - You would need to convert these numbers into their binary representations.

- 2. Separate the numbers into groups based on their radix positions:
- In the case of floating-point numbers, you can separate them into groups based on the individual bits of their binary representation. You start with the least significant bit (LSB) and move towards the most significant bit (MSB)

Radix Sort

- In general, radix sort based on counting sort is
 - Fast
 - Asymptotically fast (i.e., $O(n)$)
 - Simple to code
 - A good choice

Summary: Radix Sort

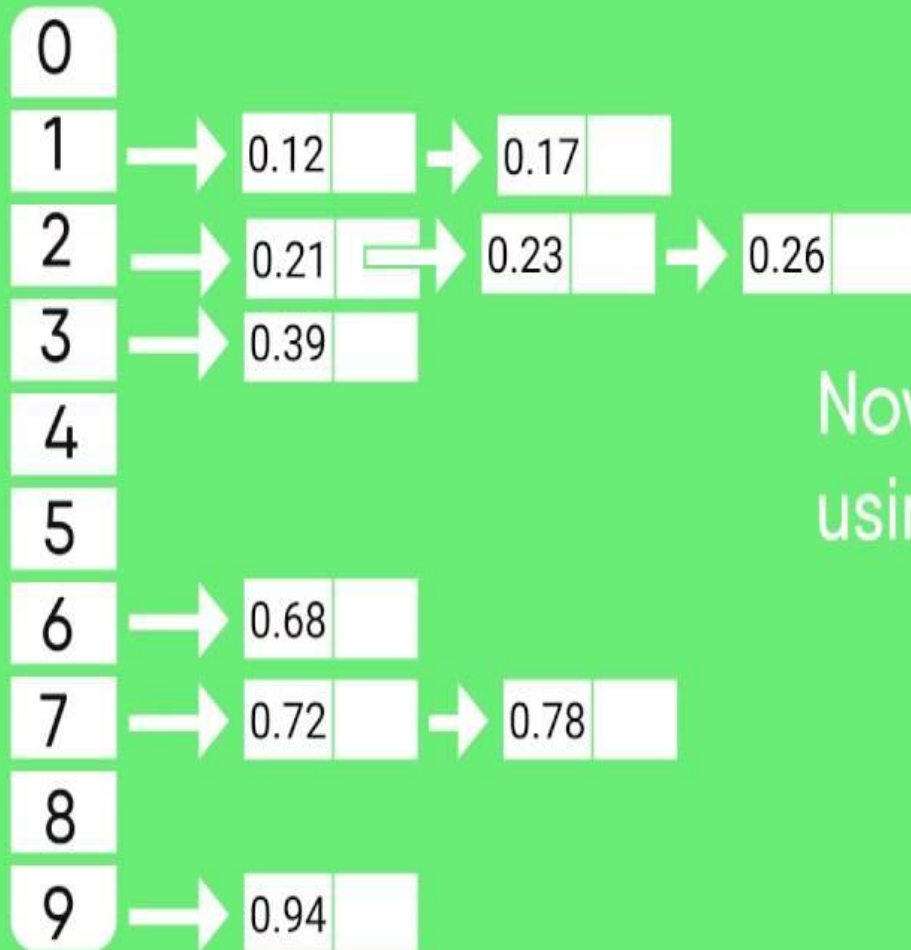
- Radix sort:
 - Assumption: input has d digits ranging from 0 to k
 - Basic idea:
 - Sort elements by digit starting with *least* significant
 - Use a stable sort (like counting sort) for each stage
 - Each pass over n numbers with d digits takes time $O(n+k)$, so total time $O(dn+dk)$
 - When d is constant and $k=O(n)$, takes $O(n)$ time
 - Fast! Stable! Simple!
 - Doesn't sort in place

Bucket Sort

□ Bucket sort

- Assumption: input is n reals from $[0, 1)$
- Basic idea:
 - Create n linked lists (*buckets*) to divide interval $[0,1)$ into subintervals of size $1/n$
 - Add each input element to appropriate bucket and sort buckets with insertion sort
- Uniform input distribution $\rightarrow O(1)$ bucket size
 - Therefore the expected total time is $O(n)$
- These idea is similar to *hash tables*

0.78 0.17 0.39 0.26 0.72 0.94 0.21 0.12 0.23 0.68



Now sort each bucket individually using insertion sort

- 
- Bucket sort is a sorting algorithm that works by distributing the elements of an array into a number of buckets or bins, and then sorting the individual buckets.
 - It is most effective when the input data is uniformly distributed across a range

Steps involved in Bucket sort

- **Determine the Range:** Find the minimum and maximum values in the input array to determine the range of values you need to sort.
- **Create Buckets:** Create an array of empty buckets, one for each possible value within the range. The number of buckets depends on the range and the distribution of data.

- 
- ❑ **Distribution:** For each element in the input array, place it into the appropriate bucket based on its value.
 - ❑ **Sort Each Bucket:** Once all elements are distributed into buckets, sort each individual bucket using a sorting algorithm.
 - ❑ You can choose any sorting algorithm for this step, but often insertion sort is used because it's efficient for small-sized lists.

-
- **Concatenate Buckets:** After sorting all the individual buckets, concatenate them together to obtain the sorted array. The order in which you concatenate the buckets depends on their indices.
 - **Final Result:** The concatenated buckets form the sorted array.

- 
- Bucket sort is efficient when the input data is uniformly distributed across the range, and the number of buckets is chosen appropriately.
 - However, its performance can degrade if the data is highly skewed or if there are many empty buckets.